



TP Escabilidad: Money Laundering Analysis

Integrante	Padrón
Carbajal Robles Kevin Emir	108304
Rea Matias	99770
Sendra Alejo	107716

Sistemas Distribuidos I (TA050)
Cátedra Roca - 1er cuatrimestre de 2026

28 de mayo de 2026

Índice

Índice.....	1
Introducción.....	2
Análisis del problema.....	3
Requerimientos funcionales (consultas).....	3
Atributos del dataset.....	3
Mapeo de atributos por consulta.....	4
Procesamiento y flujo de datos.....	5
Procesamiento de datos.....	5
Diagrama de robustez.....	7
Consulta 1.....	7
Consulta 2.....	7
Consulta 3.....	8
Consulta 4.....	8
Consulta 5.....	9
Comunicación y comportamiento.....	10
Consulta 3.....	10
Consulta 4.....	11
Flujo del sistema.....	12
Punto de Entrada.....	13
Consulta 3.....	13
Consulta 4.....	14
Organización Modular y Estructura.....	15
Diagrama de paquetes.....	15
Infraestructura y Despliegue.....	17
Tareas.....	18
Referencias.....	19

Introducción

El presente informe detalla el seguimiento, las modificaciones y detalles de implementación del diseño propuesto en el hito anterior. En este caso se busca que el sistema sea escalable soportando, al menos, un 50% de los nodos con réplicas.

Modificaciones del diseño

Respetando los requerimientos se ha modificado el diagrama de robustez para distribuir de forma adecuada las tareas de procesamiento.

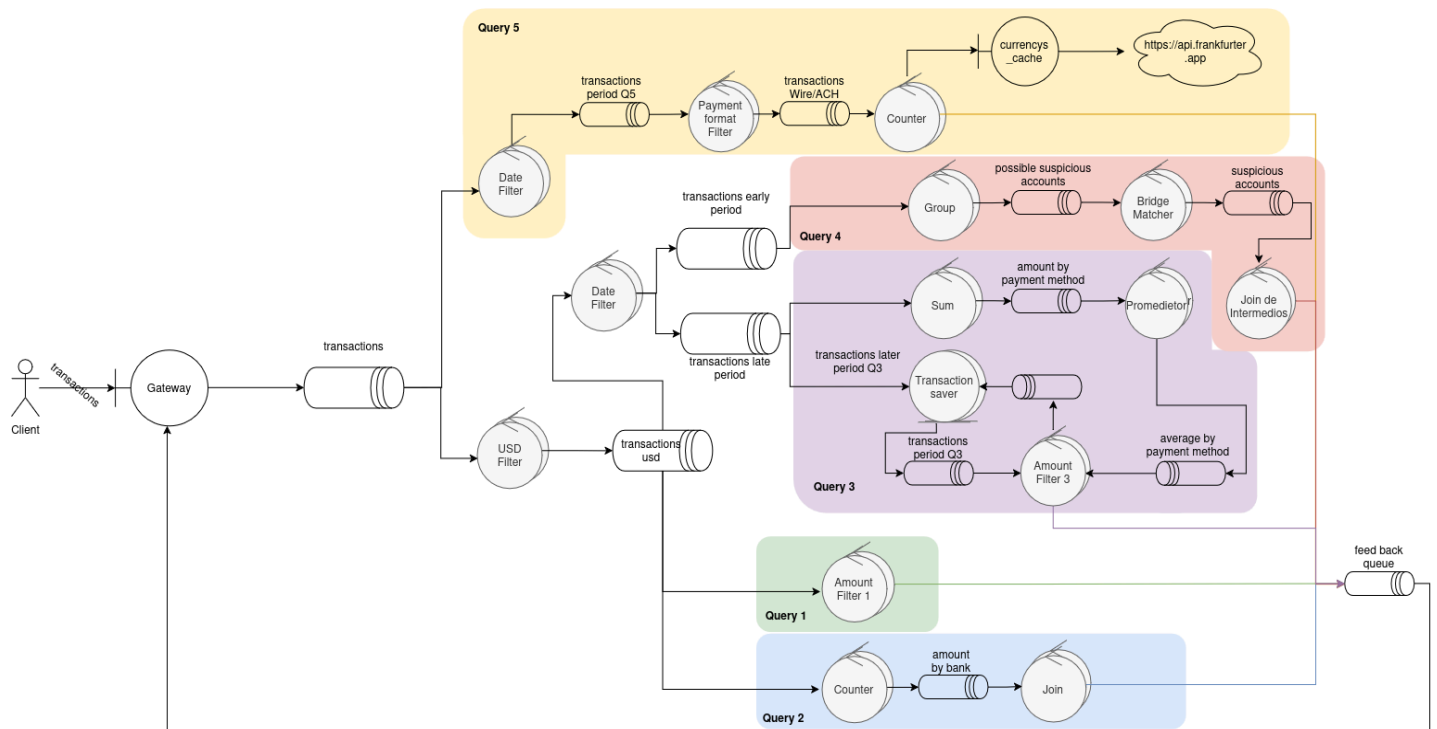


Figura 1: Diagrama de Robustez general.

Mapeo de atributos por consulta

Para todas las consultas, exceptuando la 5ta, se filtra primeramente por *Payment Currency* == "US Dollar".

Se utilizarán los siguientes campos para cada consulta:

Query 1: Transacciones USD < 50

- **Campos:** *From Bank, Account, To Bank, Account.1, Amount Paid*.
- **Uso:** Se filtra *Amount Paid* < 50 para extraer la tupla de cuenta origen, destino y monto.

Query 2: Máximo por banco

- **Campos:** *From Bank, Account, Amount Paid*.
- **Uso:** Se utiliza *From Bank* como clave de guardado. Se comparan los valores de *Amount Paid* para hallar el máximo global por cada banco.

Query 3: Desvío del promedio histórico

- **Campos:** *Timestamp, From Bank, Account, Amount Paid, Payment Format*.
- **Uso:**
 - **Periodo de promedio:** Se usa *Timestamp, Payment Format* y *Amount Paid* para calcular el promedio por formato.
 - **Periodo de detección:** Se usa *From Bank, Account* y *Amount Paid* para filtrar contra el 1% del promedio obtenido anteriormente.

Query 4: Patrón scatter-cather

- **Campos:** *Timestamp, From Bank, Account, To Bank, Account.1*.
- **Uso:** Se analizan las relaciones entre *From Bank, Account* y *To Bank, Account.1* dentro del periodo indicado.

Query 5: Micro-transferencias Wire/ACH

- **Campos clave:** *Timestamp, Payment Format, Amount Paid*.
- **Uso:** Se aplica un filtro combinado: *Timestamp* en el periodo inicial y *Payment Format* de "Wire" o "ACH". Finalmente contabilizamos aquellas transacciones con *Amount Paid* convertido (en caso de ser necesario) a USD < 1.

Procesamiento y flujo de datos

En esta sección se tiene el objetivo de desarrollar sobre la implementación del sistema, explicando la cadena de procesos por la que pasa la información, así como la comunicación entre el cliente y el sistema, y la comunicación interna.

Procesamiento de datos

A continuación, se detalla como es el procesamiento de datos para cada consulta.

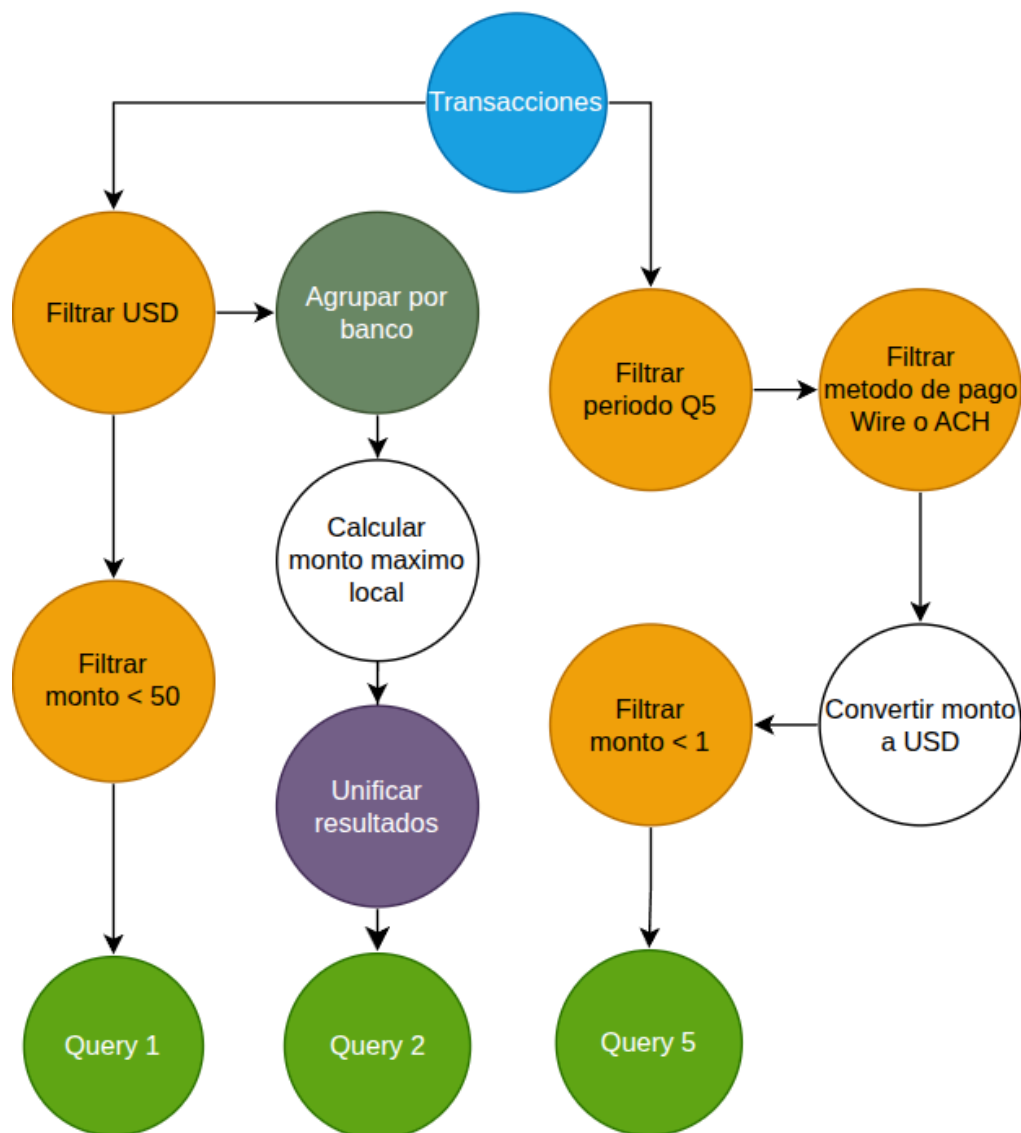


Figura 2: DAG para las consultas 1, 2 y 5

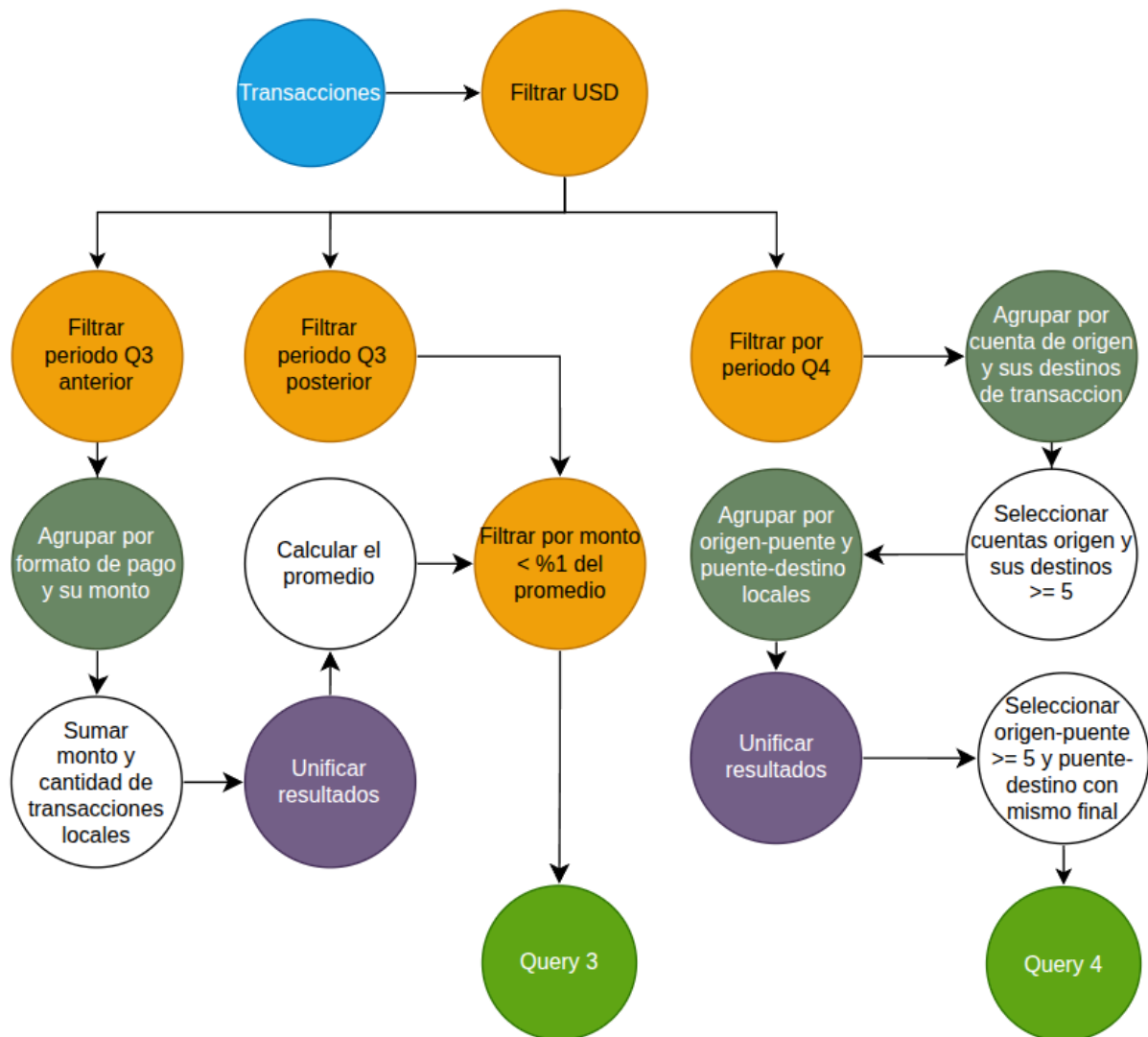


Figura 3: DAG para las consultas 3 y 4

En el diagrama se observa que el flujo principal de datos atraviesa una serie de filtros comunes, tales como el filtro de moneda USD y la segmentación por periodos temporales. A partir de estas etapas iniciales, las transacciones se distribuyen a través de diversos nodos especializados en tareas de agrupación, mapeo y agregación de datos.

Si bien el sistema optimiza el procesamiento paralelo, algunos flujos presentan una complejidad lógica superior debido a sus requerimientos:

Algunos flujos pueden resultar complejos a simple vista, como por ejemplo:

- Consulta 3: Requiere que el sistema calcule un promedio global por formato de pago en un periodo inicial. Los nodos de procesamiento posterior permanecen a la espera de este valor para habilitar el filtrado final.
- Consulta 4: Se identifica primero cuentas orígenes con una dispersión hacia 5 o más destinos. Una vez marcados como sospechosos, el sistema rastrea los movimientos de estos intermediarios para confirmar si convergen en un destino final común
- Consulta 5: Se integra un nodo de consumo de API externa. Este componente realiza la conversión de cada divisa descubierta a USD, permitiendo que la validación de umbrales sea independiente de la moneda original de la transacción.

Cada flujo de procesamiento concluye en un nodo sumidero específico para cada consulta, donde los resultados se consolidan y se retornan al Gateway para su entrega al cliente.

Diagrama de robustez - Escenarios

El punto de entrada al sistema es el Gateway, componente encargado de recibir y canalizar el flujo de transacciones enviadas por el cliente. Inicialmente, todas las operaciones siguen un flujo común de ingesta y pre-procesamiento:

- Ingesta: El cliente envía lotes de transacciones al Gateway.
- Despacho: Las transacciones se dirigen hacia un exchange desde el cual todos los nodos interesados reciben la totalidad de las transacciones enviadas por el cliente.

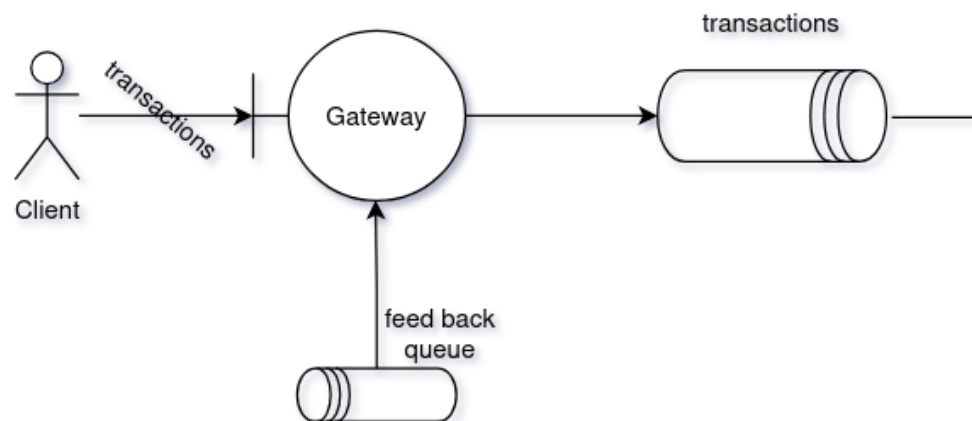


Figura 4: Flujo inicial general de todas las consultas

Consulta 1

Para la primera consulta, el sistema identifica las transacciones realizadas en dólares con un monto inferior a 50. Una vez filtradas, la cuenta de origen, el destino y el monto se retornan al Gateway a través de una cola de mensajes para su consolidación final. Una vez un worker “Amount Filter 1” recibe aviso del EOR, por parte del usuario,

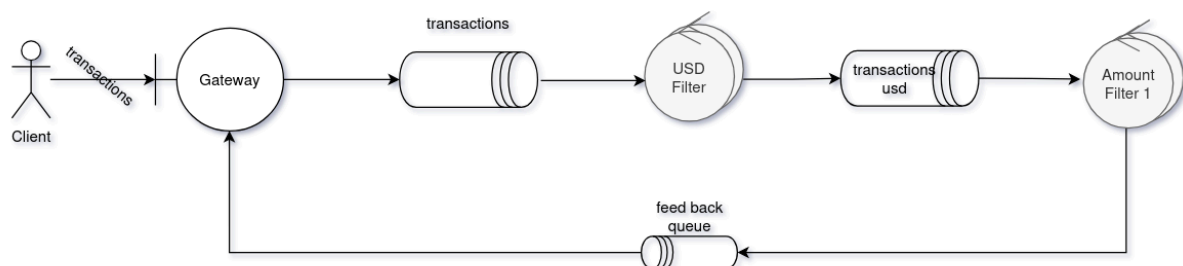


Figura 5: Flujo de la consulta 1

Consulta 2

En este escenario, se filtran las transacciones en USD para luego agruparlas por entidad bancaria. Se calcula el monto máximo local en cada nodo de procesamiento Counter. Una vez que el cliente haya terminado de enviar, se unifican los resultados parciales en el Join para determinar el máximo global por banco, devolviendo la información (Banco, Cuenta y Monto) al Gateway.

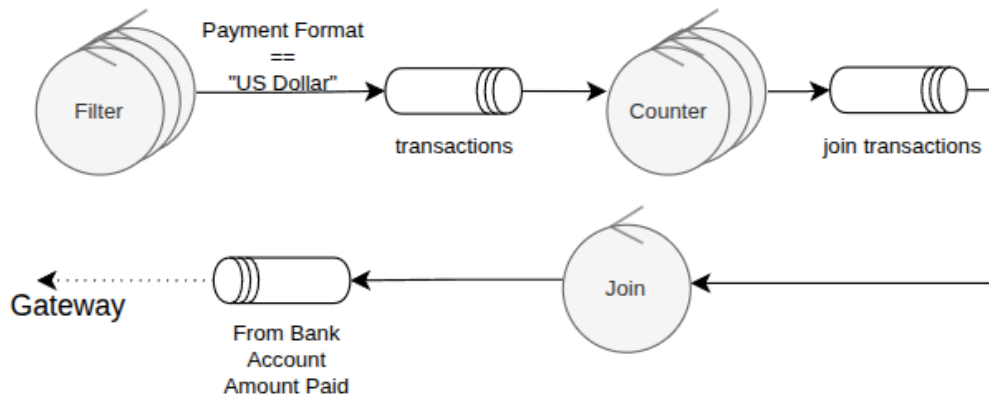


Figura 6: Flujo de la consulta 2

Consulta 3

La consulta se divide en dos flujos, donde el segundo depende del primero. Los filters iniciales filtran las transacciones en USD.

1. Periodo 1/9/2022 al 5/9/2022: se suman los montos por tipo de transacción y la cantidad. Luego el nodo join “average” calcula el promedio de cada una y le envía los resultados al segundo flujo.
2. Periodo 6/9/2022 a 15/9/2022: mientras no tenga los resultados de los promedios los nodos filter guardan los datos en los storage. Al recibir la notificación con los promedios, filtra las transacciones que no superen el centésimo por tipo.

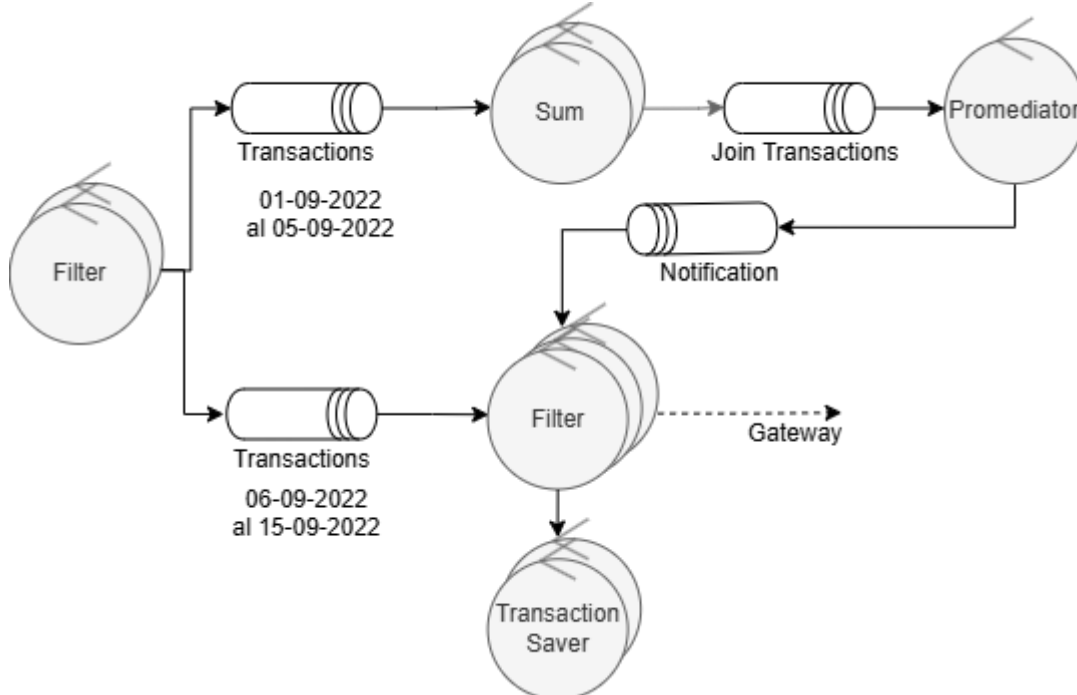


Figura 7: Flujo de la consulta 3

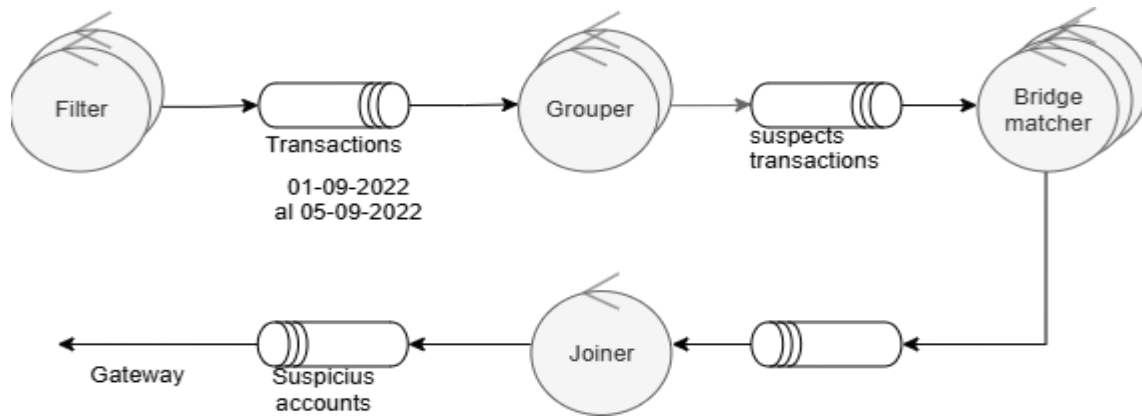
Consulta 4

El flujo comienza obteniendo las transacciones que se realizaron en el periodo 1-9-22 al 5-9-22 y sean en USD.

Luego se las agrupan por (origen - n cuentas destino) y (destino - n cuentas origen), de esta forma creamos los patrones Fan-out y Fan-in. Cada grouper contendrá toda la información para de un origen o destino en particular ya que el envío depende del hash de estos valores.

El Bridge matcher comprueba si se cumple un camino desde una cuenta A a una B por distintas cuentas intermedias.

El joiner recopila los resultados de los distintos nodos y lo envía al gateway



Consulta 5

Para la quinta consulta, los Filters detectan inicialmente aquellas transacciones entre los periodos [2022-09-01, 2022-09-05] y con método de pago "Wire" o "ACH". Luego, filtran aquellas con montos menores a 1 USD. Debido a la variedad de divisas en el dataset, cada filter se comunica con una API externa para obtener las cotizaciones, y convertir el umbral de "1 USD" a la moneda correspondiente (quote) de la transacción.

Para minimizar las peticiones de red, los nodos de filtrado implementan un mecanismo de caché local de conversiones. De esta manera, si una cotización ya fue consultada por algún nodo, los demás pueden reutilizar dicha información para procesar transacciones similares.

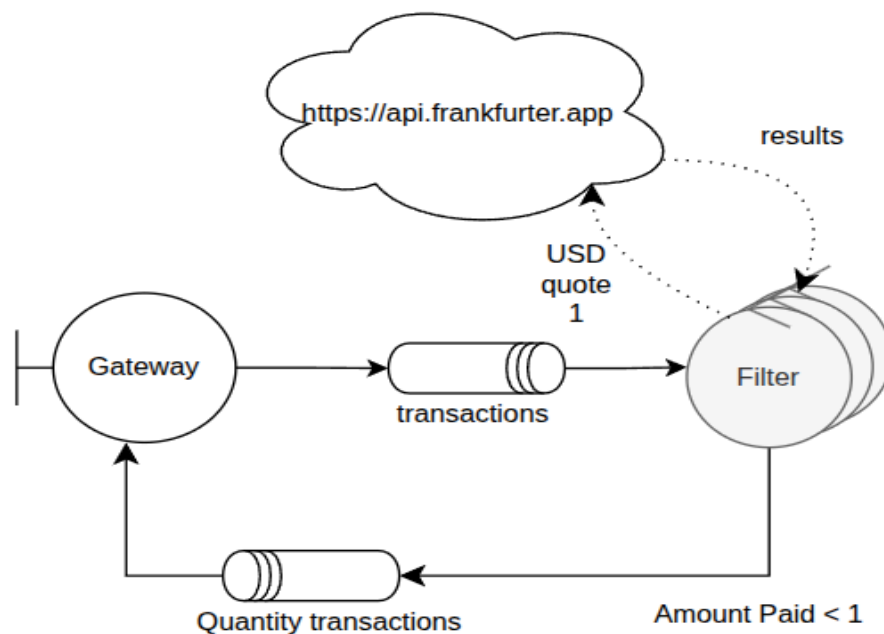


Figura 9: Flujo de la consulta 5

Comunicación y comportamiento

Para mayor comprensión sobre cómo se comunican los nodos, y cómo estos se comportan, en la presente sección se desarrollan algunos diagramas para ilustrar su comportamiento.

En el caso general todas consultas comienzan como ilustra el siguiente diagrama:

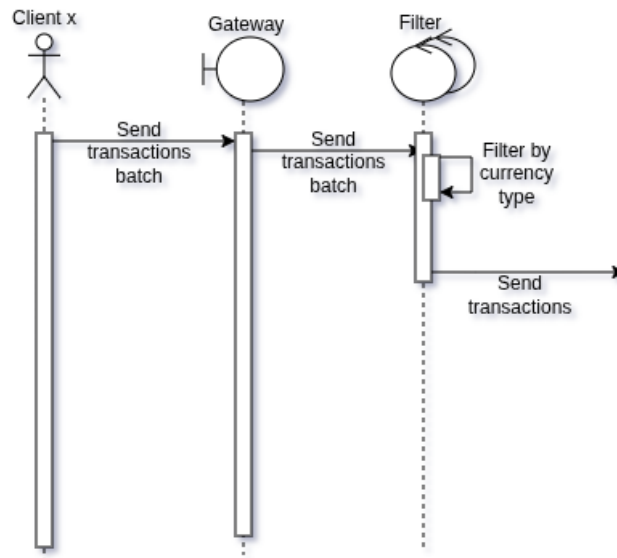


Figura 10: Diagrama de secuencia ilustrando comienzo general de las secuencias

Como se observa en el diagrama, toda transacción será enviada al filter y este mismo se encargará de replicar el dato en las queues correspondientes, ya sea para ser considerada en los conteos o como sospechosa ante el cumplimiento de los patrones anteriormente indicados.

A continuación se desarrollan los diagramas de secuencia de los casos más complejos a resolver por el sistema.

Consulta 3

Como se observa en la figura, el filter, además de filtrar por el tipo de moneda se encargará de encolar las consultas según si dicha transacción cumple con las fechas estipuladas por lo impuesto en la consulta 3 o 4.

Posteriormente al filtrado, se ha considerado persistir en disco la información de las transacciones más actuales dado que se requiere el promedio de las transacciones más antiguas para procesarlas.

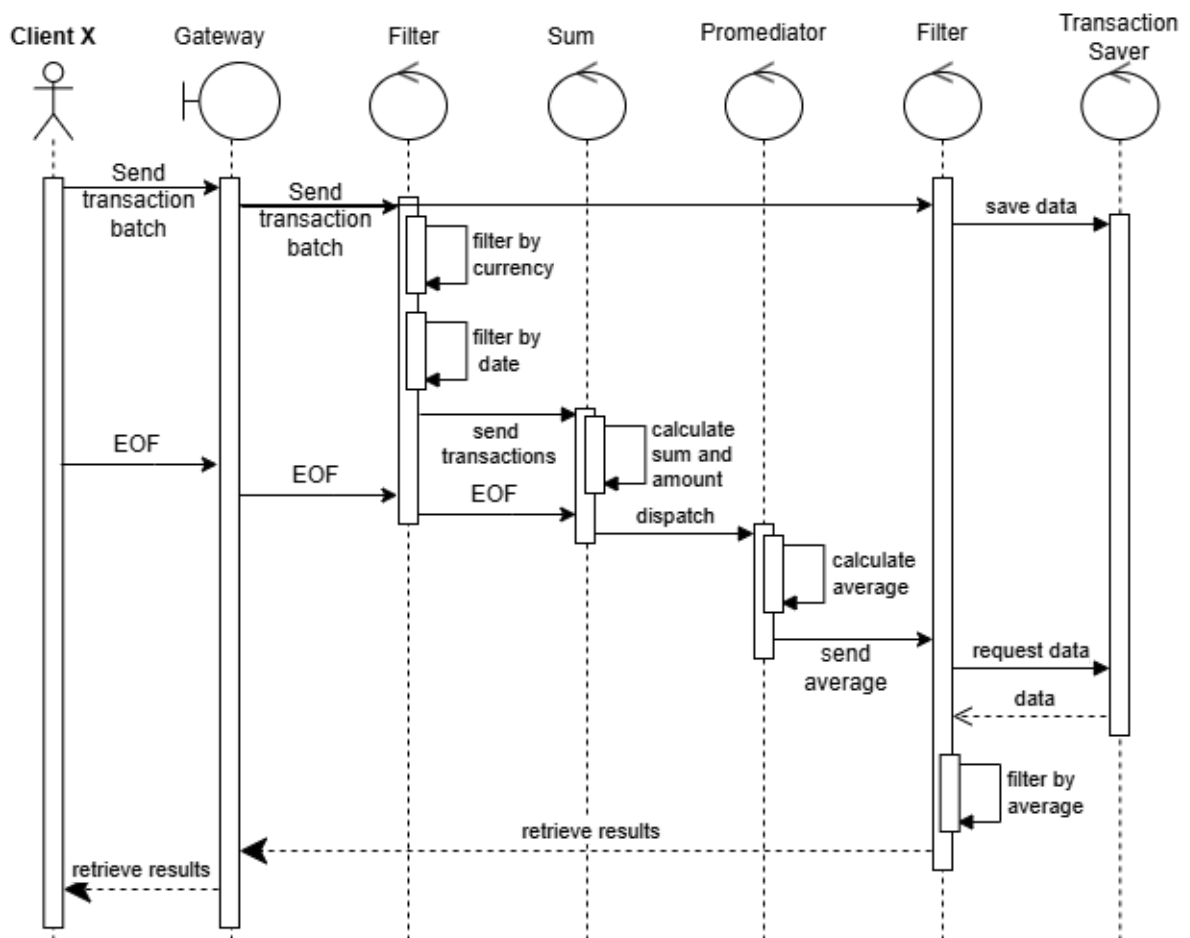


Figura 11: Diagrama de secuencia que ilustra la secuencia de operaciones a llevar a cabo para la resolución de la consulta 3.

Consulta 4

Analicemos ahora el caso 4, donde se requiere de identificar un patrón en las transacciones de forma que se debe identificar las cuentas de origen que cumplan con el patrón scatter gather con solo una cuenta intermedia.

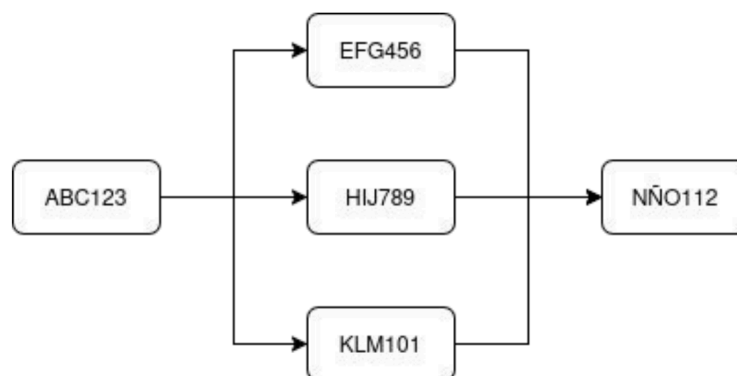


Figura 12: Ilustración del patrón scatter-gather como un grafo dirigido.

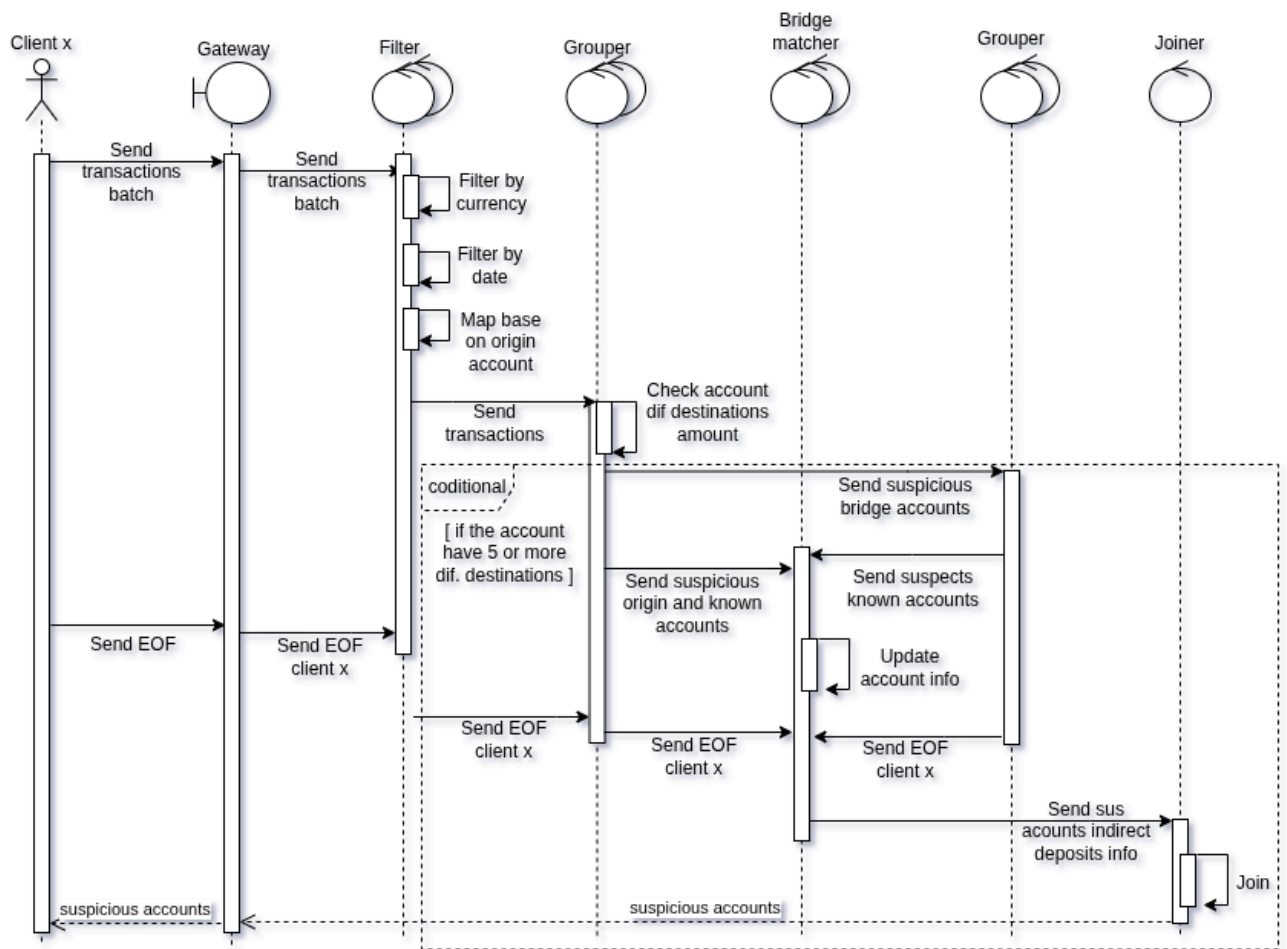


Figura 13: Diagrama de secuencia que ilustra la secuencia de operaciones a llevar a cabo para la resolución de la consulta 4.

Para la resolución de esta consulta se plantea implementar una estrategia de particionamiento (sharding) basada en la cuenta de origen para garantizar el determinismo en la distribución de carga, no solo con el objetivo de distribuir la carga, sino también permitiendo definir de forma determinista el mismo nodo de destino para las transacciones con mismo origen, simplificando así la lógica necesaria para sincronizar la información. Solo cuando se detecta un posible sospechoso las diferentes copias de “group” se comunican entre sí para notificar las posibles cuentas sospechosas de ser puentes entre el origen, y destino, sospechosos de lavado de activos.

Flujo del sistema

Veamos ahora las diferentes actividades desarrolladas por los diferentes actores del sistema en diferentes escenarios.

Punto de Entrada

En principio, observamos el flujo entre los nodos en el punto de entrada al sistema para así comprender mejor las actividades desarrolladas previo a la resolución de las consultas.

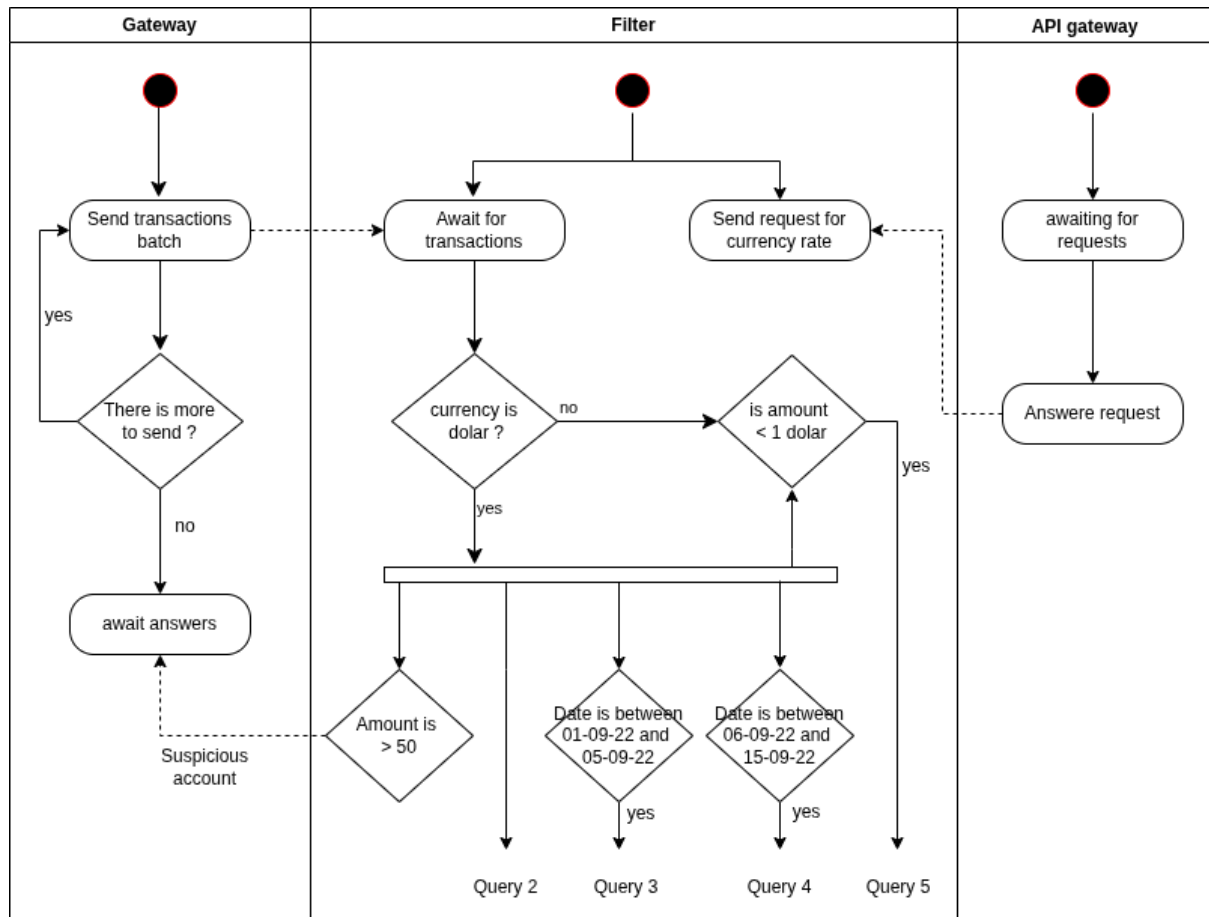


Figura 14: Diagrama de actividades indicando las actividades desarrolladas en el punto de entrada al sistema propuesto.

Mientras el gateway recibe los batches de transferencias del usuario este los deriva al filter quien se encargará de distribuir las transferencias, en la medida que estas cumplan las condiciones asociadas, a las diferentes queues donde los workers correspondientes se encargaran de procesarlas.

Consulta 3

Observemos ahora el caso 3 donde contamos con dos flujos de transferencias divididos a ser procesados para resolver una única consulta.

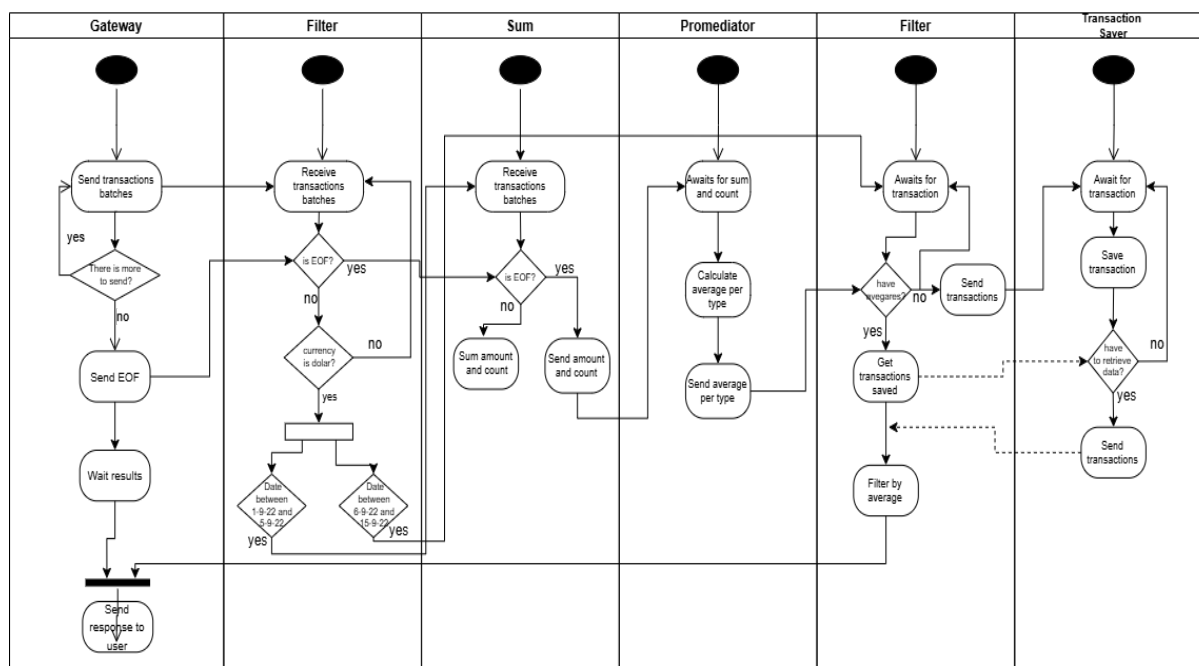


Figura 15: Diagrama de actividades en seguimiento del desarrollo de la consulta 3.

Del diagrama podemos observar que en una primera fase, el primer filter se encarga de dividir las transferencias en base a la fecha de realización. De esta manera, hasta que los nodos de Sum no reciben el EOF por parte del cliente estos calculan el amount y el count para los distintos tipos de transferencias.

Cuando reciben el EOF, le envían sus resultados parciales al Promediator. Una vez calculado el promedio el nodo correspondiente notifica a la entidad encargada de procesar los posibles casos de lavado.

El filter encargado le pide al transaction saver que le devuelva, si es que tiene, las transferencias que fue guardando al no poder procesar sin el promedio. Este procede a diferenciar los casos de lavado ya sea desde transacciones que le pueden llegar desde el filter original o las persistidas.

Al finalizar le envía los resultados al gateway para devolverle al usuario los resultados.

Consulta 4

En optimización a la primera propuesta planteada se ha implementado, para la resolución de la query 4 un sistema de workers “group” encargados de rutear las transacciones a un “Bridge Matcher” determinado (mediante sharding en base al origen de la transacción) para que este lleve conteo de las transacciones enviadas por cada cuenta y notifique a los workers de la siguiente etapa sobre la existencia de esta posible cuenta sospechosa y posteriormente envíe toda la información al join.

Para indicar la finalización todos los nodos group deben enviar el mensaje EOR a cada uno de los “Bridge Matchers” con el objetivo de dar por finalizada la recepción de transacciones por determinado usuario. En la fase de “Bridge Matchers” estos se notifican entre sí mediante un 2 fase commit con motivo de confirmar la finalización del análisis de las transacciones (esto se cumple dado el prefetch 1 explicado en las observaciones)

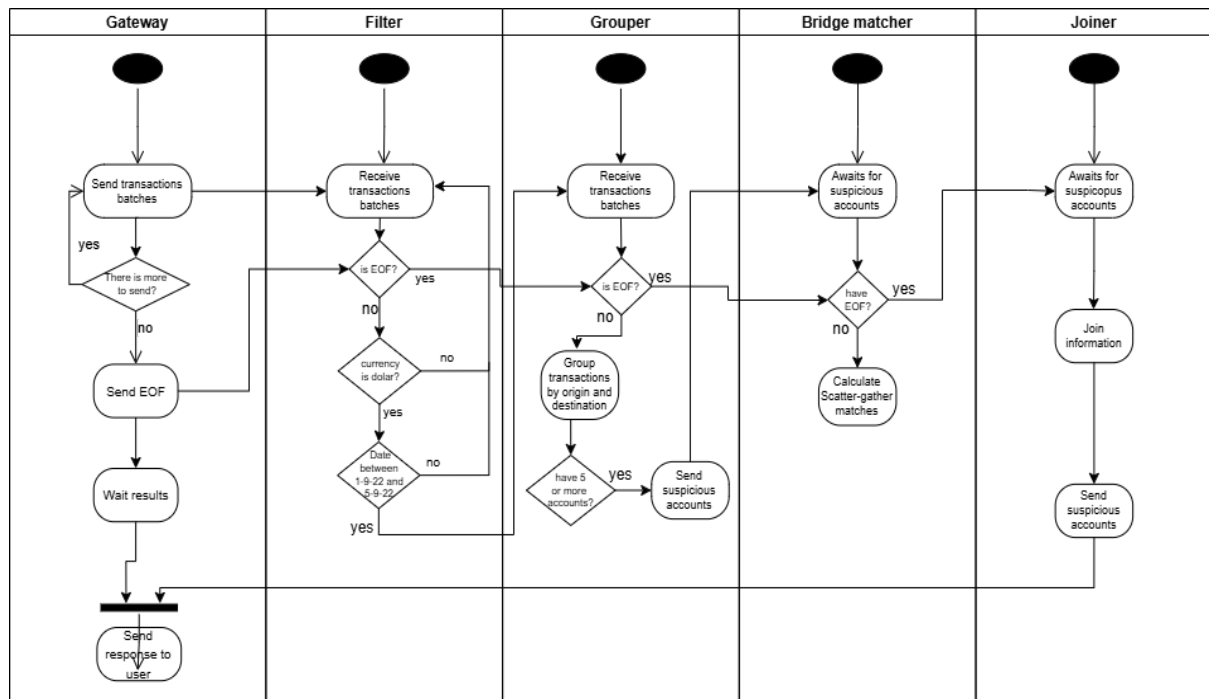


Figura 16: Diagrama de actividades del desarrollo de la consulta 4.

Organización Modular y Estructura

Diagrama de paquetes

Se muestran los diagramas de la estructura del sistema con respecto a sus módulos y paquetes, como así las dependencias entre ellas.

El paquete common centraliza las definiciones compartidas por todos los nodos, garantizando que tanto el Gateway como los Workers interpreten el formato de las transacciones y los mensajes del middleware bajo un mismo estándar

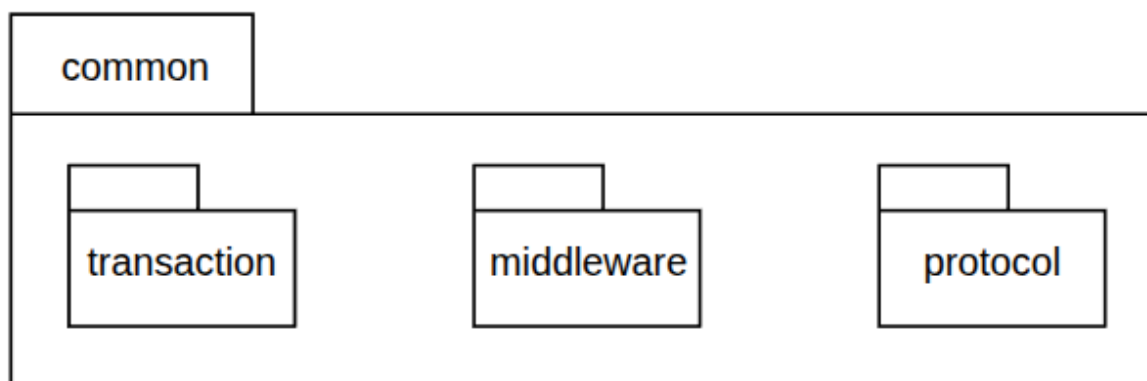


Figura 17: Paquete que tienen en común los demas

Los módulos de los workers van a ser muy parecidos entre sí, tal así que se lo puede agrupar de la siguiente manera:

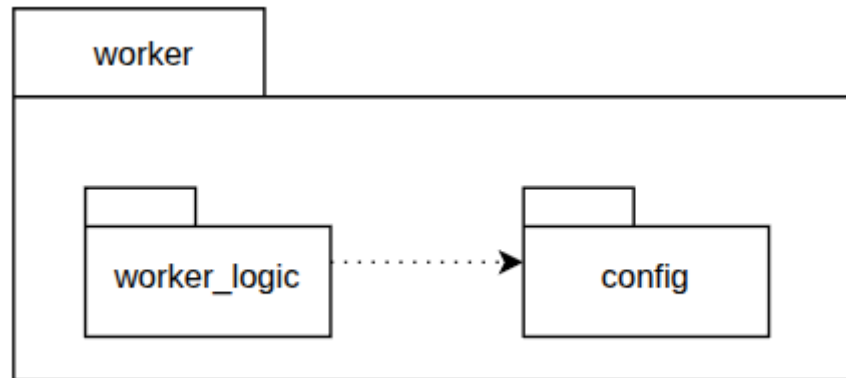


Figura 18: Paquete del worker

Con worker nos referimos a toda instancia replicable del sistema que ejecuta una tarea de cómputo específica, como el Group, Bridge Matcher, Sum, Counter y Transaction Saver.

A continuación, se detalla la estructura que van a tener el Gateway y el Filter.

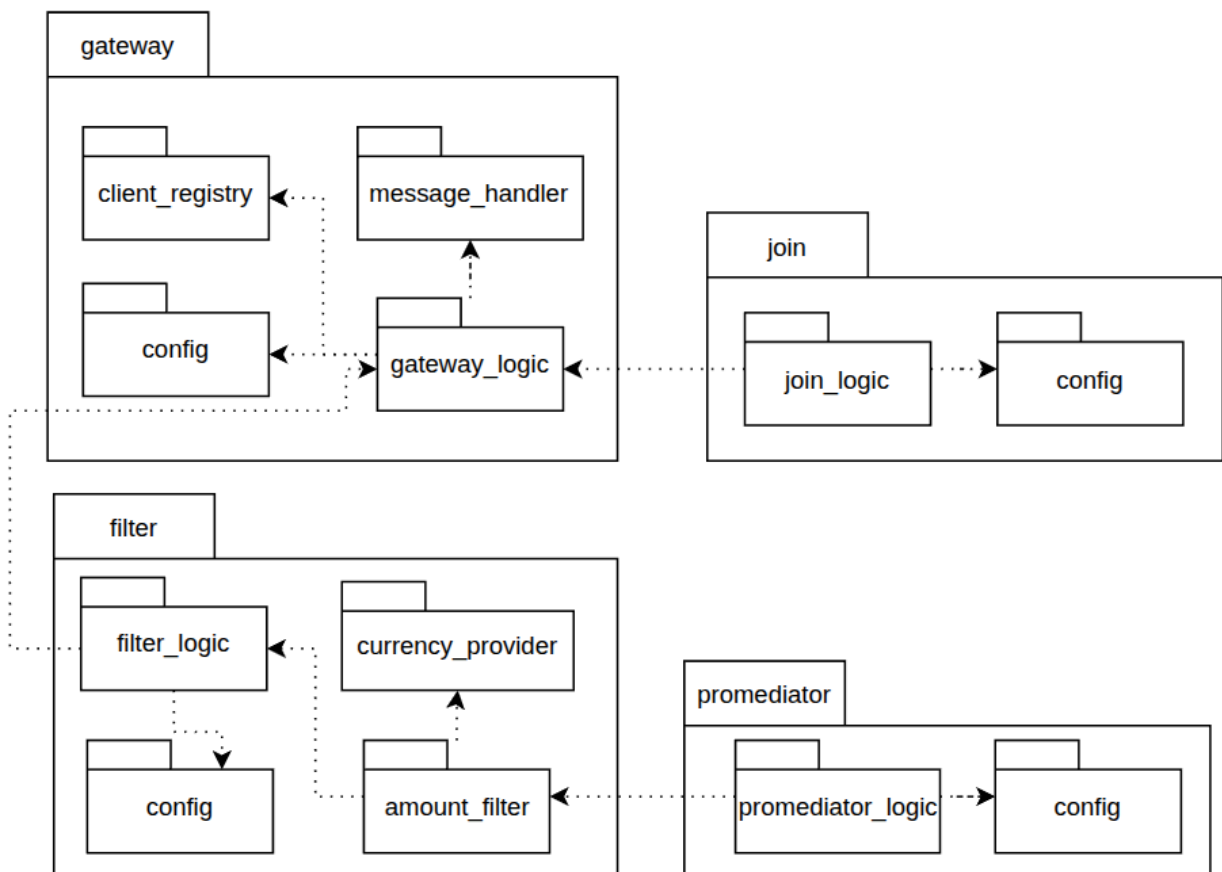


Figura 19: Paquete del gateway, filter, join y promediator

- Gateway: Funciona como el punto de enlace (interfaz) del sistema. Se encarga de la gestión de clientes (client_registry) y del orquestamiento de mensajes hacia los filtros mediante gateway_logic.

- Filter: Se encarga del pre-procesamiento encargado del despacho por periodos temporales y divisas. Integra el currency_provider para la normalización de montos y el amount_filter para la ejecución de umbrales en las consultas 1, 3 y 5.
- Join y Promediator: El primero reconstruye las relaciones del patrón scatter-gather, mientras que el segundo realiza la agregación para el cálculo de promedios temporales.

Notar que cada paquete cuenta con una config independiente. Eso permite que cada nodo gestione sus propios parámetros de red y variables de entorno de forma aislada.

Infraestructura y Despliegue

A continuación se detalla la infraestructura ideada para el funcionamiento distribuido del sistema.

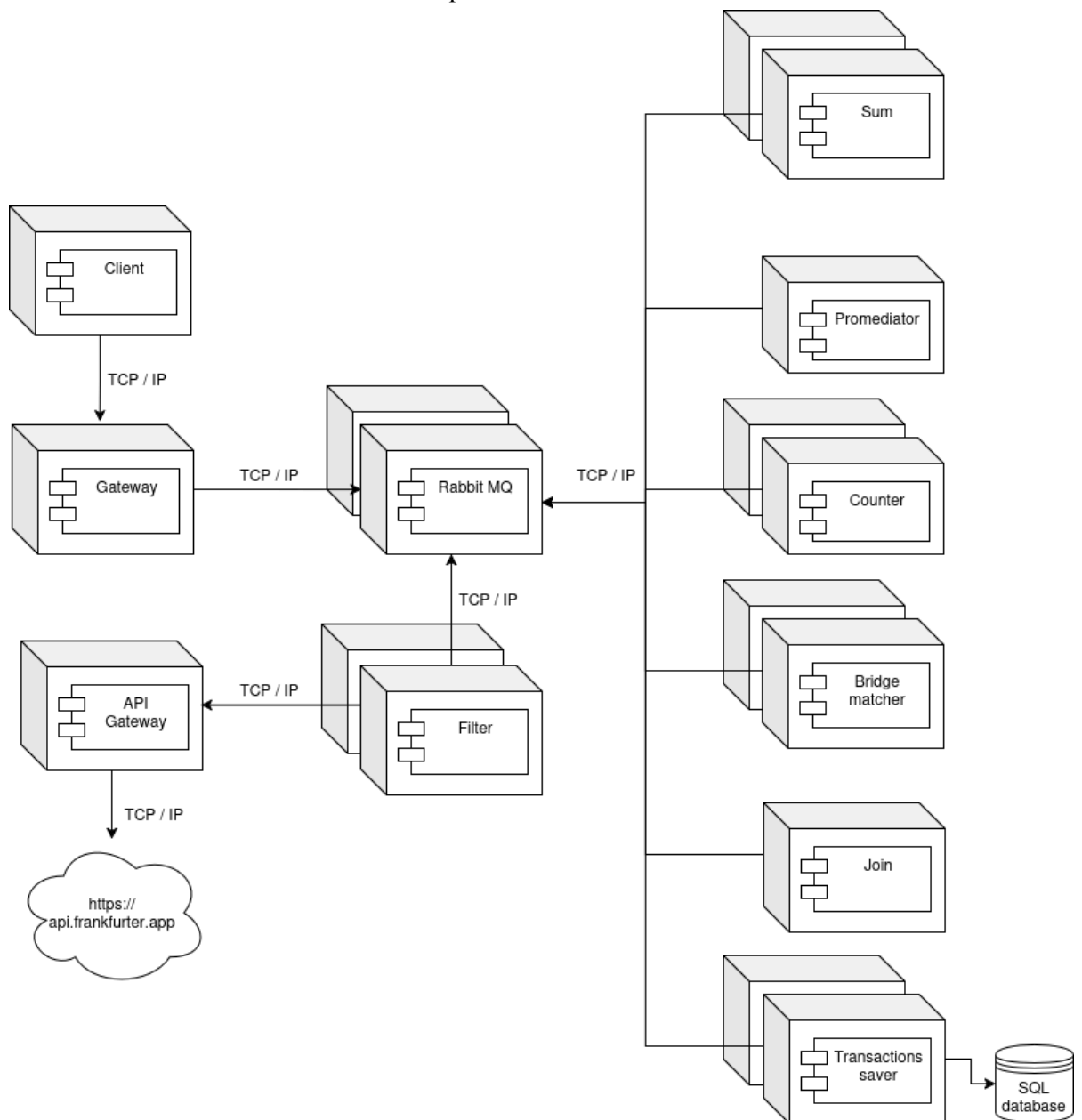


Figura 20: Diagrama de despliegue del sistema propuesto.

Características de la arquitectura:

- Para la comunicación entre nodos internos del sistema, se definen las queues de RabbitMQ como medio de comunicación y distribución de trabajo como también como medio de notificación general mediante broadcast según lo requiera la lógica de sincronización entre nodos.
- El filtrado de datos, entiéndase reducción de columnas o resolución de consultas básicas, es realizado por el filtro en el ingreso al sistema, de esta manera se reduce la magnitud de los datos circulantes y se da una respuesta temprana a la primera consulta, reduciendo el tiempo ocioso del cliente y del gateway.

Consideraciones

- El sistema cuenta con una única instancia del nodo “Gateway” como entrada al sistema, siendo este un único punto de falla para el cual no se han considerado contramedidas aún.
- Para el desarrollo del trabajo práctico se ha hecho uso de la feature “prefetch” , particularidad del middleware “RabbitMQ”, con valor igual a uno, de forma que los trabajadores reciben los mensajes de a uno por vez, permitiendo recibir los mensajes en el orden en que fueron enviados al middleware simplificando la consideración del desorden y la complejidad de coordinar los múltiples nodos. Sin embargo, esta feature presenta la desventaja de reducir notablemente la velocidad a la que se reciben los mensajes.
- Pese a lo propuesto, y expresado en el diagrama de robustez, se presentó gran dificultad para resolver la query 4 de forma que no se logró replicar los nodos “join”, siendo este otro “único punto de falla”. Así mismo, los resultados finales dados por el conjunto de workers, respectivos a la Query 4, no resultó igual al propuesto por la cátedra.

Referencias

- [1] Altman, E. (s.f.). IBM Transactions for Anti Money Laundering (AML) [Dataset]. Kaggle. <https://www.kaggle.com/datasets/ealtman2019/ibm-transactions-for-anti-money-laundering-aml>
- [2] Frankfurter. (s.f.). *Frankfurter API: Foreign exchange rates API*. <https://api.frankfurter.dev/>
- [3] pablotroca. (s.f.). Money laundering analysis [Notebook]. Kaggle. <https://www.kaggle.com/code/pablotroca/money-laundering-analysis>